

complete list of C#, .NET, ASP.NET MVC, ASP.NET Core interview + non-interview coding questions covering 2025 & 2026—all levels (Beginner → Advanced → System Design).

 SECTION A — C# Coding Questions (Beginner → Expert)

✓ Basic C# Coding

1. Reverse a string without using built-in function
2. Check palindrome string
3. Count vowels and consonants in a string
4. Find duplicate characters in a string
5. Swap 2 numbers without temp variable
6. Fibonacci series
7. Prime number check
8. Factorial recursion
9. Check Armstrong number
10. Find second largest number in array

✓ Intermediate C# Coding

11. Find missing number in an array
12. Count occurrence of each char using Dictionary
13. LINQ: Group employees by Department
14. LINQ: Get top 3 highest salaries
15. Convert JSON to C# object and vice-versa
16. Read/write file using FileStream
17. Multi-threading example (Task, Thread, Parallel)
18. Async/Await demo with real API call
19. Events & Delegates simple program
20. SOLID example (SRP, OCP) using small classes

✓ Advanced C# Coding

21. Custom exception handling example
22. Implement IDisposable
23. Write your own generic repository
24. Custom attribute + reflection example
25. Lock, Monitor, Mutex, Semaphore example

- 26. Memory optimization using Span<T>
 - 27. Channels in .NET example
 - 28. Minimal API + records example (.NET 8/9)
 - 29. Caching: MemoryCache example
 - 30. Best way to handle configuration using IOptions
-

SECTION B — ASP.NET MVC Coding

- 31. Create a custom ActionFilter
 - 32. Model validation with DataAnnotations
 - 33. Upload file to server
 - 34. Export data to Excel in MVC
 - 35. Custom HTML helper method
 - 36. Repository pattern with EF
 - 37. Partial View + AJAX example
 - 38. JQuery DataTable binding
 - 39. Generate PDF in MVC
 - 40. Global exception filter
-

SECTION C — ASP.NET Core Coding (Real-Time + Interview Focus)

Beginner Core

- 41. Build a full CRUD API using Entity Framework Core
- 42. Dependency Injection basic example
- 43. Logging using SeriLog / built-in logger
- 44. Model binding & validation
- 45. Middleware example (custom logging middleware)

Intermediate Core

- 46. JWT Authentication & Authorization
- 47. Refresh token implementation
- 48. Versioning API example
- 49. Rate limiting (ASP.NET Core 8 built-in)
- 50. File upload + streaming in Core

- 51. Global exception handling middleware
- 52. Health checks (database + API)
- 53. Background hosted service
- 54. Send emails using SendGrid/SMTP
- 55. CORS configuration

✓ Advanced Core (2025/2026 questions)

- 56. Minimal APIs with filters
- 57. API performance optimization
- 58. Distributed caching (Redis example)
- 59. CQRS pattern with MediatR
- 60. Clean Architecture folder structure
- 61. Onion Architecture example
- 62. WebSockets in ASP.NET Core
- 63. SignalR real-time dashboard
- 64. gRPC service creation
- 65. Multi-tenant API design
- 66. Audit logging with Action Filters
- 67. Upload large files with streaming
- 68. API gateway using YARP
- 69. Feature flags implementation
- 70. Worker Service + Cron jobs (.NET 8/9)

** SECTION D — LINQ & EF Core Coding**

- 71. LINQ queries for filtering, grouping
- 72. EF Core migrations
- 73. Query splitting in EF Core
- 74. Projection queries with Select
- 75. Soft delete implementation
- 76. Bulk insert using EF Core
- 77. Track vs NoTracking example
- 78. Custom conventions

- 79. EF Core Interceptors
 - 80. Concurrency Handling (RowVersion)
-

SECTION E — System Design (For Senior Developer Interviews 2025/2026)

- 81. Design a billing system API
 - 82. Design an inventory management system
 - 83. Design real-time chat using SignalR
 - 84. Design notification service (Email + SMS)
 - 85. Design high performance report API
 - 86. Design file upload + virus scan service
 - 87. Design token authentication service
 - 88. Design a multi-tenant SaaS application
 - 89. Event-driven architecture using RabbitMQ
 - 90. Distributed caching design
-

SECTION F — DotNet Advanced & Real-World Coding

- 91. Clean Architecture with CQRS complete sample
 - 92. API rate limiting with policies
 - 93. Implement retry policies using Polly
 - 94. Upload files to Azure Blob
 - 95. Read secrets from Azure KeyVault
 - 96. API unit testing (xUnit + Moq)
 - 97. Integration testing in ASP.NET Core
 - 98. Build custom NuGet package
 - 99. Swagger documentation customization
 - 100. Code review checklist for .NET
-

SECTION G — Non-Interview Real-Project Tasks

- 101. Implement Invoice PDF generation
- 102. Scheduled daily email reports
- 103. Auto clean old logs using hosted service

C#, .NET, ASP.NET MVC, ASP.NET Core interview + non-interview coding questions

- 104. Excel import (OpenXML)
- 105. Excel export (OpenXML)
- 106. Token-based login page
- 107. Billing system modules design
- 108. User/Role Permission Management
- 109. File versioning & storage system
- 110. Print-ready PDF layout for invoices

SECTION A — C# Coding (FULL for items 1–30)

Create a new Console App (dotnet new console -n CSharpSamples). Each example can be used in Main or separate files. I'll present each example as a small class/program you can run.

1. Reverse a string without using built-in function

using System;

```
class ReverseStringExample {  
    static string Reverse(string s) {  
        char[] arr = new char[s.Length];  
        for(int i=0;i<s.Length;i++){  
            arr[i] = s[s.Length-1-i];  
        }  
        return new string(arr);  
    }  
  
    static void Main() {  
        string input = "hello";  
        Console.WriteLine(Reverse(input)); // "olleh"  
    }  
}
```

2. Check palindrome string

using System;

```
class Palindrome {  
    static bool IsPalindrome(string s) {  
        int i=0, j=s.Length-1;  
        while(i<j) {  
            if (char.ToLower(s[i]) != char.ToLower(s[j])) return false;  
            i++; j--;  
        }  
        return true;  
    }  
}
```

```
}  
  
static void Main() {  
  
    Console.WriteLine(IsPalindrome("Madam")); // True  
  
}  
}
```

3. Count vowels and consonants in a string

```
using System;  
  
using System.Linq;  
  
class CountVowelsConsonants {  
  
    static void Main() {  
  
        string s = "Hello World!";  
  
        var vowels = "aeiou";  
  
        int v = s.ToLower().Count(c => vowels.Contains(c));  
  
        int consonants = s.ToLower().Count(c => char.IsLetter(c) && !vowels.Contains(c));  
  
        Console.WriteLine($"Vowels: {v}, Consonants: {consonants}");  
  
    }  
}
```

4. Find duplicate characters in a string

```
using System;  
  
using System.Linq;  
  
using System.Collections.Generic;  
  
class Dups {  
  
    static void Main() {  
  
        string s = "programming";  
  
        var dup = s.GroupBy(c => c)  
  
            .Where(g => g.Count() > 1)  
  
            .Select(g => new { Char = g.Key, Count = g.Count() });  
  
        foreach(var d in dup) Console.WriteLine($"{d.Char} => {d.Count}");  
  
    }  
}
```

5. Swap two numbers without temp variable

using System;

```
class Swap {  
    static void Main() {  
        int a = 5, b = 7;  
        a = a + b; // 12  
        b = a - b; // 5  
        a = a - b; // 7  
        Console.WriteLine($"{a}, {b}"); // 7,5  
        // OR XOR trick for ints  
    }  
}
```

6. Fibonacci series

using System;

```
class Fibonacci {  
    static void Main() {  
        int n = 10;  
        int a=0,b=1;  
        Console.Write($"{a} {b} ");  
        for(int i=2;i<n;i++){  
            int c = a+b;  
            Console.Write($"{c} ");  
            a=b; b=c;  
        }  
    }  
}
```

7. Prime number check

using System;


```
class Prime {  
    static bool IsPrime(int n) {  
        if (n < 2) return false;  
        if (n % 2 == 0) return n == 2;  
        int r = (int)Math.Sqrt(n);  
        for(int i=3;i<=r;i+=2) if (n % i == 0) return false;  
        return true;  
    }  
    static void Main() {  
        Console.WriteLine(IsPrime(29)); // True  
    }  
}
```

8. Factorial recursion

```
using System;  
class Fact {  
    static long Factorial(int n) => n <= 1 ? 1 : n * Factorial(n-1);  
    static void Main() {  
        Console.WriteLine(Factorial(10));  
    }  
}
```

9. Check Armstrong number

```
using System;  
class Armstrong {  
    static bool IsArmstrong(int n) {  
        var s = n.ToString();  
        int power = s.Length;  
        int sum = 0;  
        foreach(char c in s) sum += (int)Math.Pow(c - '0', power);  
        return sum == n;  
    }  
    static void Main() {
```

```
        Console.WriteLine(IsArmstrong(153)); // True
    }
}
```

10. Find second largest number in array

```
using System;

using System.Linq;

class SecondLargest {

    static int SecondLargestNum(int[] arr) {

        return arr.Distinct().OrderByDescending(x => x).Skip(1).First();

    }

    static void Main() {

        int[] arr = {2,5,1,5,3};

        Console.WriteLine(SecondLargestNum(arr)); // 3

    }

}
```

11. Find missing number in an array (1..N)

```
using System;

class Missing {

    static int FindMissing(int[] arr, int n) {

        int sum = n*(n+1)/2;

        int actual = 0;

        foreach(var x in arr) actual += x;

        return sum - actual;

    }

    static void Main() {

        int[] arr = {1,2,4,5};

        Console.WriteLine(FindMissing(arr,5)); // 3

    }

}
```

12. Count occurrence of each char using Dictionary

```
using System;

using System.Collections.Generic;

class CharCount {

    static void Main() {

        string s = "banana";

        var dict = new Dictionary<char,int>();

        foreach(var c in s) {

            if(dict.ContainsKey(c)) dict[c]++;

            else dict[c] = 1;

        }

        foreach(var kv in dict) Console.WriteLine($"{kv.Key} => {kv.Value}");

    }

}
```

13. LINQ: Group employees by Department

```
using System;

using System.Linq;

using System.Collections.Generic;

class Employee { public string Name; public string Dept; }

class LINQGroup {

    static void Main() {

        var list = new [] {

            new Employee {Name="A", Dept="HR"},

            new Employee {Name="B", Dept="Dev"},

            new Employee {Name="C", Dept="Dev"}

        };

        var grouped = list.GroupBy(e => e.Dept);

        foreach(var g in grouped) {

            Console.WriteLine(g.Key + ": " + string.Join(", ", g.Select(e => e.Name)));

        }

    }

}
```

```
}  
}
```

14. LINQ: Get top 3 highest salaries

```
using System;  
using System.Linq;  
class Salaries {  
    static void Main() {  
        var salaries = new[]{ 1000, 5000, 7000, 4000, 7000 };  
        var top3 = salaries.OrderByDescending(x => x).Distinct().Take(3);  
        Console.WriteLine(string.Join(", ", top3));  
    }  
}
```

15. Convert JSON to C# object and vice-versa (System.Text.Json)

```
using System;  
using System.Text.Json;  
  
class Person { public string Name {get;set;} public int Age {get;set;} }  
  
class JsonExample {  
    static void Main() {  
        var p = new Person { Name="Ibrahim", Age=30 };  
        string json = JsonSerializer.Serialize(p);  
        Console.WriteLine(json);  
        var des = JsonSerializer.Deserialize<Person>(json);  
        Console.WriteLine(des.Name);  
    }  
}
```

16. Read/write file using FileStream

```
using System;  
using System.IO;  
class FileStreamExample {
```

```
static void Main() {  
    string path = "test.txt";  
    using(var fs = new FileStream(path, FileMode.Create, FileAccess.Write)) {  
        var bytes = System.Text.Encoding.UTF8.GetBytes("Hello FileStream");  
        fs.Write(bytes,0,bytes.Length);  
    }  
    Console.WriteLine(File.ReadAllText(path));  
}
```

17. Multi-threading example (Task, Thread, Parallel)

```
using System;  
using System.Threading;  
using System.Threading.Tasks;  
  
class Threading {  
    static void Main() {  
        Task.Run(() => {  
            Thread.Sleep(500);  
            Console.WriteLine("Task completed");  
        }).Wait();  
  
        Thread t = new Thread(() => Console.WriteLine("Thread running"));  
        t.Start();  
        t.Join();  
  
        Parallel.For(0,5,i => Console.WriteLine($"Parallel {i}"));  
    }  
}
```

18. Async/Await demo (simulate API call)

```
using System;  
using System.Threading.Tasks;
```

```
class AsyncDemo {  
    static async Task<string> FakeApiCall() {  
        await Task.Delay(500);  
        return "result";  
    }  
    static async Task Main() {  
        var res = await FakeApiCall();  
        Console.WriteLine(res);  
    }  
}
```

19. Events & Delegates simple program

```
using System;
```

```
delegate void Notify(string message);
```

```
class Publisher {  
    public event Notify OnNotify;  
    public void DoSomething() {  
        OnNotify?.Invoke("Done");  
    }  
}
```

```
class EventExample {  
    static void Main() {  
        var p = new Publisher();  
        p.OnNotify += (msg) => Console.WriteLine($"Received {msg}");  
        p.DoSomething();  
    }  
}
```

20. SOLID example (SRP, OCP) using small classes

```
// SRP: separate responsibilities
```

```
class Invoice { public decimal Amount {get;set;} }
```

C#, .NET, ASP.NET MVC, ASP.NET Core interview + non-interview coding questions

```
class InvoicePrinter { public void Print(Invoice i) => Console.WriteLine($"Invoice {i.Amount}"); }
```

```
class InvoiceRepository { public void Save(Invoice i) { /*persist*/ } }
```

// OCP: extendable via interface

```
interface ITaxCalculator { decimal CalcTax(Invoice i); }
```

```
class BasicTax: ITaxCalculator { public decimal CalcTax(Invoice i) => i.Amount*0.1m; }
```

(Short explanation: each class has single responsibility; new tax calculators added without modifying existing code.)

21. Custom exception handling example

```
using System;
```

```
class MyException : Exception {
```

```
    public MyException(string message): base(message) {}
```

```
}
```

```
class CustomExceptionExample {
```

```
    static void Test(int x) {
```

```
        if (x < 0) throw new MyException("Negative not allowed");
```

```
    }
```

```
    static void Main() {
```

```
        try {
```

```
            Test(-1);
```

```
        } catch(MyException ex) {
```

```
            Console.WriteLine("Caught: " + ex.Message);
```

```
        }
```

```
    }
```

```
}
```

22. Implement IDisposable

```
using System;
```

```
class ResourceHolder : IDisposable {
```

```
    private bool disposed = false;
```

```
    public void Use() {
```

```
        if(disposed) throw new ObjectDisposedException(nameof(ResourceHolder));
```

```
        Console.WriteLine("Using resource");
    }

    public void Dispose() {
        if(!disposed) {
            // free unmanaged resources

            disposed = true;

            GC.SuppressFinalize(this);
        }
    }

    ~ResourceHolder() => Dispose();
}

class DisposeExample {
    static void Main() {
        using(var r = new ResourceHolder()) r.Use();
    }
}
```

23. Write your own generic repository (basic)

```
using System;
using System.Collections.Generic;
using System.Linq;

interface IRepository<T> where T: class {
    IEnumerable<T> GetAll();

    T GetById(int id);

    void Add(T item);

    void Remove(T item);
}

class InMemoryRepository<T> : IRepository<T> where T : class {
    protected List<T> _items = new();
}
```



```
public IEnumerable<T> GetAll() => _items;

public T GetById(int id) => null; // example placeholder

public void Add(T item) => _items.Add(item);

public void Remove(T item) => _items.Remove(item);

}
```

(Real EF Core repository would implement DbContext operations; see later EF Core section.)

24. Custom attribute + reflection example

```
using System;
```

```
using System.Reflection;
```

```
[AttributeUsage(AttributeTargets.Class)]
```

```
class MyInfoAttribute : Attribute { public string Info; public MyInfoAttribute(string info) => Info = info;
}
```

```
[MyInfo("Important")]
```

```
class Sample {}
```

```
class AttrExample {
```

```
    static void Main() {
```

```
        var attr = (MyInfoAttribute)Attribute.GetCustomAttribute(typeof(Sample),
typeof(MyInfoAttribute));
```

```
        Console.WriteLine(attr.Info);
```

```
    }
```

```
}
```

25. Lock, Monitor, Mutex, Semaphore example (lock + SemaphoreSlim)

```
using System;
```

```
using System.Threading;
```

```
using System.Threading.Tasks;
```

```
class Concurrency {
```

```
    static object _lock = new object();
```

```
    static SemaphoreSlim sem = new SemaphoreSlim(2);
```

```
static void Main() {  
    Task.WaitAll(  
        Task.Run(() => { lock(_lock) Console.WriteLine("Locked section"); } ),  
        Task.Run(async () => {  
            await sem.WaitAsync();  
            Console.WriteLine("Semaphore enter");  
            sem.Release();  
        })  
    );  
}  
}
```

26. Memory optimization using Span<T>

```
using System;  
  
class SpanExample {  
    static void Main() {  
        Span<int> span = stackalloc int[5] {1,2,3,4,5};  
        for(int i=0;i<span.Length;i++) Console.WriteLine(span[i]);  
    }  
}
```

27. Channels in .NET example

```
using System;  
using System.Threading.Channels;  
using System.Threading.Tasks;  
  
class ChannelExample {  
    static async Task Main() {  
        var channel = Channel.CreateUnbounded<int>();  
        var writer = channel.Writer;  
        var reader = channel.Reader;
```

```
var producer = Task.Run(async () => {  
    for(int i=0;i<5;i++){  
        await writer.WriteAsync(i);  
        await Task.Delay(100);  
    }  
    writer.Complete();  
});
```

```
var consumer = Task.Run(async () => {  
    await foreach(var item in reader.ReadAllAsync()) {  
        Console.WriteLine(item);  
    }  
});
```

```
    await Task.WhenAll(producer, consumer);  
}  
}
```

28. Minimal API + records example (.NET 8/9)

// Program.cs (minimal api)

using Microsoft.AspNetCore.Builder;

using Microsoft.AspNetCore.Http;

var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

record Product(int Id, string Name);

var products = new List<Product> { new(1,"Pen"), new(2,"Pencil") };

app.MapGet("/products", () => products);

```
app.MapGet("/products/{id}", (int id) => products.FirstOrDefault(p => p.Id == id) is Product p ?
Results.Ok(p) : Results.NotFound());
```

```
app.Run();
```

29. Caching: MemoryCache example

```
using Microsoft.Extensions.Caching.Memory;
```

```
using Microsoft.Extensions.DependencyInjection;
```

```
var services = new ServiceCollection();
```

```
services.AddMemoryCache();
```

```
var sp = services.BuildServiceProvider();
```

```
var cache = sp.GetRequiredService<IMemoryCache>();
```

```
cache.Set("now", DateTime.UtcNow, TimeSpan.FromSeconds(10));
```

```
Console.WriteLine(cache.Get<DateTime>("now"));
```

30. Handle configuration using IOptions

```
// appsettings.json
```

```
// { "MySettings": { "Title": "Hello" } }
```

```
// Program.cs minimal
```

```
using Microsoft.Extensions.Options;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services.Configure<MySettings>(builder.Configuration.GetSection("MySettings"));
```

```
var app = builder.Build();
```

```
app.MapGet("/", (IOptions<MySettings> opts) => opts.Value.Title);
```

```
app.Run();
```

```
public class MySettings { public string Title {get;set;} }
```

🔥 SECTION B — ASP.NET MVC Coding (COMPLETE)

Contains **15 essential coding problems** asked in interviews + real-project scenarios.

✓ **Works in ASP.NET Core MVC**

Run using:

```
dotnet new mvc -n MvcDemo
```

✅ 1. Display Student List (Controller → Service → View)

Model

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string City { get; set; }
}
```

Service

```
public interface IStudentService
{
    List<Student> GetAll();
}
```

```
public class StudentService : IStudentService
{
    public List<Student> GetAll() =>
    {
        new()
        {
            new Student { Id = 1, Name = "Ibrahim", City = "Mumbai" },
            new Student { Id = 2, Name = "Ahmed", City = "Pune" }
        };
    }
}
```

Controller

```
public class StudentsController : Controller
```

```
{  
    private readonly IStudentService _service;  
  
    public StudentsController(IStudentService service)  
    {  
        _service = service;  
    }  
  
    public IActionResult Index()  
    {  
        var students = _service.GetAll();  
        return View(students);  
    }  
}
```

View (Index.cshtml)

@model List<Student>

```
<table class="table table-bordered">  
    <tr><th>ID</th><th>Name</th><th>City</th></tr>  
    @foreach (var s in Model)  
    {  
        <tr>  
            <td>@s.Id</td>  
            <td>@s.Name</td>  
            <td>@s.City</td>  
        </tr>  
    }  
</table>
```

✅ 2. Submit Form (Create Student)

Controller

[HttpGet]

```
public IActionResult Create() => View();
```

[HttpPost]

```
public IActionResult Create(Student model)
```

```
{
```

```
    if (!ModelState.IsValid)
```

```
        return View(model);
```

```
    TempData["msg"] = $"Student {model.Name} saved!";
```

```
    return RedirectToAction("Index");
```

```
}
```

View (Create.cshtml)

```
<form asp-action="Create" method="post" class="form">
```

```
    <div>
```

```
        <label>Name</label>
```

```
        <input asp-for="Name" class="form-control"/>
```

```
        <span asp-validation-for="Name" class="text-danger"></span>
```

```
    </div>
```

```
    <div>
```

```
        <label>City</label>
```

```
        <input asp-for="City" class="form-control"/>
```

```
    </div>
```

```
    <button type="submit" class="btn btn-primary mt-2">Save</button>
```

```
</form>
```

✅ 3. Edit Student (GET + POST)

Controller

[HttpGet]

```
public IActionResult Edit(int id)
{
    var student = _service.GetAll().FirstOrDefault(x => x.Id == id);
    return View(student);
}
```

[HttpPost]

```
public IActionResult Edit(Student model)
{
    // Update logic...
    TempData["msg"] = "Updated!";
    return RedirectToAction("Index");
}
```

4. Delete Student

Controller

```
public IActionResult Delete(int id)
{
    TempData["msg"] = $"Deleted student {id}";
    return RedirectToAction("Index");
}
```

5. Search + Filter

Controller

```
public IActionResult Search(string name)
{
    var list = _service.GetAll()
        .Where(x => string.IsNullOrEmpty(name) ||
            x.Name.Contains(name))
}
```



```
.ToList();
```

```
return View(list);
```

```
}
```

View

```
<form method="get">
```

```
    <input type="text" name="name" placeholder="Search..." class="form-control"/>
```

```
    <button class="btn btn-info">Search</button>
```

```
</form>
```

```
@foreach (var s in Model)
```

```
{
```

```
    <p>@s.Name - @s.City</p>
```

```
}
```

✅ 6. Dropdown Binding (Cities)

Controller

```
public IActionResult CityFilter()
```

```
{
```

```
    ViewBag.Cities = new SelectList(new[] { "Mumbai", "Pune", "Delhi" });
```

```
    return View();
```

```
}
```

View

```
<select asp-items="ViewBag.Cities" class="form-control"></select>
```

✅ 7. Upload File in MVC

Controller

```
[HttpPost]
```

```
public async Task<IActionResult> Upload(IFormFile file)
```

```
{
```

```
    if (file != null)
```

```
{  
    string path = Path.Combine("wwwroot/uploads", file.FileName);  
    using var stream = new FileStream(path, FileMode.Create);  
    await file.CopyToAsync(stream);  
}  
TempData["msg"] = "File uploaded successfully";  
return RedirectToAction("Index");  
}
```

View

```
<form asp-action="Upload" method="post" enctype="multipart/form-data">  
    <input type="file" name="file"/>  
    <button class="btn btn-primary">Upload</button>  
</form>
```

8. Partial View Example

Controller

```
public IActionResult _StudentCard(Student s)  
{  
    return PartialView(s);  
}
```

Partial View (_StudentCard.cshtml)

```
<div class="card p-3 mb-2">  
    <h4>@Model.Name</h4>  
    <p>@Model.City</p>  
</div>
```

Use inside main view

```
@foreach (var s in Model)  
{  
    @await Html.PartialAsync("_StudentCard", s)  
}
```

✅ 9. AJAX (jQuery) Insert Student

Controller

```
[HttpPost]

public IActionResult AjaxInsert(Student model)

{

    return Json(new { status = "success", student = model });

}
```

View

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

```
<input id="name"/>
```

```
<button id="btn">Add</button>
```

```
<script>
```

```
$("#btn").click(function () {

    $.post("/Students/AjaxInsert", { Name: $("#name").val() }, function (res) {

        alert("Inserted: " + res.student.name);

    });

});

</script>
```

✅ 10. Return JSON in MVC

```
public IActionResult GetJson()

{

    return Json(new { name = "Ibrahim", city = "Mumbai" });

}
```

✅ 11. Login + Session Example

Controller

```
[HttpPost]
public IActionResult Login(string username, string password)
{
    if (username == "admin" && password == "123")
    {
        HttpContext.Session.SetString("user", username);
        return RedirectToAction("Dashboard");
    }

    ViewBag.error = "Invalid Login";
    return View();
}

public IActionResult Dashboard()
{
    var user = HttpContext.Session.GetString("user");
    if (user == null) return RedirectToAction("Login");
    return Content($"Welcome {user}");
}
```

12. Custom Attribute (Action Filter)

Attribute

```
public class LogActionFilter : ActionFilterAttribute
{
    public override void OnActionExecuted(ActionExecutedContext context)
    {
        Console.WriteLine("Action Executed: " + context.ActionDescriptor.DisplayName);
    }
}
```

Use

```
[LogActionFilter]
```

```
public IActionResult Index() => View();
```

✓ 13. TempData Usage

```
public IActionResult Save()
{
    TempData["msg"] = "Saved successfully!";
    return RedirectToAction("Index");
}
```

```
public IActionResult Index()
{
    ViewBag.Message = TempData["msg"];
    return View();
}
```

✓ 14. Strongly Typed ViewModel With Multiple Lists

ViewModel

```
public class DashboardVM
{
    public List<Student> Students { get; set; }
    public List<string> Cities { get; set; }
}
```

Controller

```
public IActionResult Dashboard()
{
    return View(new DashboardVM
    {
        Students = _service.GetAll(),
        Cities = new List<string> { "Mumbai", "Delhi" }
    });
}
```

View

@model DashboardVM

```
<h3>Students</h3>
```

```
@foreach(var s in Model.Students)
```

```
{
```

```
    <p>@s.Name</p>
```

```
}
```

```
<h3>Cities</h3>
```

```
@foreach(var c in Model.Cities)
```

```
{
```

```
    <p>@c</p>
```

```
}
```

✅ 15. Custom Route Example

Route in Controller

```
[Route("student/details/{id}")]
```

```
public IActionResult Details(int id)
```

```
{
```

```
    return Content($"Details of student {id}");
```

```
}
```

SECTION C — ASP.NET Core Coding (Real-Time + Interview Focus)

All code uses .NET 8 style.

BEGINNER ASP.NET CORE

1 Build a Full CRUD API using Entity Framework Core

✓ Question

Create a CRUD API for Student using EF Core Code-First.

✓ Answer + Code

1. Model

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
}
```

2. DbContext

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) {}

    public DbSet<Student> Students { get; set; }
}
```

3. Program.cs

```
builder.Services.AddDbContext<AppDbContext>(opt =>
    opt.UseSqlServer(builder.Configuration.GetConnectionString("Default")));
builder.Services.AddControllers();

var app = builder.Build();
app.MapControllers();
```

```
app.Run();
```

4. Controller

```
[ApiController]
```

```
[Route("api/[controller]")]
```

```
public class StudentsController : ControllerBase
```

```
{
```

```
    private readonly ApplicationDbContext _ctx;
```

```
    public StudentsController(ApplicationDbContext ctx) => _ctx = ctx;
```

```
    [HttpGet]
```

```
    public async Task<IActionResult> GetAll() =>
```

```
        Ok(await _ctx.Students.ToListAsync());
```

```
    [HttpGet("{id}")]
```

```
    public async Task<IActionResult> Get(int id)
```

```
    {
```

```
        var student = await _ctx.Students.FindAsync(id);
```

```
        if (student == null) return NotFound();
```

```
        return Ok(student);
```

```
    }
```

```
    [HttpPost]
```

```
    public async Task<IActionResult> Create(Student s)
```

```
    {
```

```
        _ctx.Students.Add(s);
```

```
        await _ctx.SaveChangesAsync();
```

```
        return CreatedAtAction(nameof(Get), new { id = s.Id }, s);
```

```
    }
```

```
    [HttpPut("{id}")]
```

```
    public async Task<IActionResult> Update(int id, Student s)
```



```
{
    if (id != s.Id) return BadRequest();
    _ctx.Entry(s).State = EntityState.Modified;
    await _ctx.SaveChangesAsync();
    return NoContent();
}

[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    var s = await _ctx.Students.FindAsync(id);
    if (s == null) return NotFound();
    _ctx.Remove(s);
    await _ctx.SaveChangesAsync();
    return NoContent();
}
}
```

2 Dependency Injection — Basic Example

✓ Code

Service Interface

```
public interface IMessageService
{
    string GetMessage();
}
```

Implementation

```
public class MessageService : IMessageService
{
    public string GetMessage() => "Hello from DI Service!";
}
```

Program.cs

```
builder.Services.AddScoped<IMessageService, MessageService>();
```

Controller

```
[ApiController]
[Route("api/di")]
public class DIController : ControllerBase
{
    private readonly IMessageService _svc;

    public DIController(IMessageService svc) => _svc = svc;

    [HttpGet]
    public string GetMsg() => _svc.GetMessage();
}
```

3 Logging using Serilog

✓ Program.cs

```
Log.Logger = new LoggerConfiguration()
    .WriteTo.Console()
    .WriteTo.File("logs/log.txt")
    .CreateLogger();
```

```
builder.Host.UseSerilog();
```

Use in Controller

```
private readonly ILogger<HomeController> _log;

public HomeController(ILogger<HomeController> log) => _log = log;

_log.LogInformation("API Called!");
```

4 Model Binding & Validation

Model

```
public class RegisterDto
{
```

[Required]

```
public string Name { get; set; }
```

[EmailAddress]

```
public string Email { get; set; }
```

```
}
```

Controller

[HttpPost]

```
public IActionResult Register(RegisterDto dto)
```

```
{
```

```
    if (!ModelState.IsValid) return BadRequest(ModelState);
```

```
    return Ok("Success");
```

```
}
```

5 Custom Middleware

✓ Logging Middleware

```
public class RequestLogMiddleware
```

```
{
```

```
    private readonly RequestDelegate _next;
```

```
    public RequestLogMiddleware(RequestDelegate next) => _next = next;
```

```
    public async Task Invoke(HttpContext context)
```

```
    {
```

```
        Console.WriteLine($"Request → {context.Request.Path}");
```

```
        await _next(context);
```

```
    }
```

```
}
```

Program.cs

```
app.UseMiddleware<RequestLogMiddleware>();
```

✓ INTERMEDIATE ASP.NET CORE

6 JWT Authentication & Authorization

✓ Add Authentication

```
builder.Services.AddAuthentication("Bearer")
    .AddJwtBearer(opt =>
    {
        opt.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = false,
            ValidateAudience = false,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes("super-secret-key-123"))
        };
    });
```

✓ Generate Token

```
var key = Encoding.UTF8.GetBytes("super-secret-key-123");
var token = new JwtSecurityToken(
    claims: new[] { new Claim("email", "test@mail.com") },
    expires: DateTime.UtcNow.AddHours(1),
    signingCredentials: new SigningCredentials(
        new SymmetricSecurityKey(key), SecurityAlgorithms.HmacSha256)
);

var jwt = new JwtSecurityTokenHandler().WriteToken(token);
return jwt;
```

7 Refresh Token Implementation

✓ Model

```
public class RefreshToken
{
```

```
public string Token { get; set; }  
public DateTime Expiry { get; set; }  
}
```

✓ Generate

```
string newRefreshToken = Convert.ToBase64String(RandomNumberGenerator.GetBytes(64));
```

8 API Versioning

Program.cs

```
builder.Services.AddApiVersioning(o =>  
{  
    o.DefaultApiVersion = new ApiVersion(1, 0);  
    o.AssumeDefaultVersionWhenUnspecified = true;  
});
```

Controller

```
[ApiVersion("1.0")]  
[Route("api/v{version:apiVersion}/products")]  
public class ProductsV1Controller : ControllerBase { }
```

9 Rate Limiting (ASP.NET Core 8)

Program.cs

```
builder.Services.AddRateLimiter(o =>  
{  
    o.AddFixedWindowLimiter("fixed", opt =>  
    {  
        opt.PermitLimit = 5;  
        opt.Window = TimeSpan.FromSeconds(10);  
    });  
});  
app.UseRateLimiter();
```

10 File Upload + Streaming

Controller

```
[HttpPost("upload")]
public async Task<ActionResult> Upload(IFormFile file)
{
    using var stream = new FileStream($"uploads/{file.FileName}", FileMode.Create);
    await file.CopyToAsync(stream);
    return Ok("Uploaded");
}
```

1 1 Global Exception Handling

Middleware

```
public class GlobalExceptionMiddleware
{
    private readonly RequestDelegate _next;

    public GlobalExceptionMiddleware(RequestDelegate next) => _next = next;

    public async Task Invoke(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (Exception ex)
        {
            context.Response.StatusCode = 500;
            await context.Response.WriteAsync(ex.Message);
        }
    }
}
```

Program.cs

```
app.UseMiddleware<GlobalExceptionMiddleware>();
```

1 2 Health Checks

```
builder.Services.AddHealthChecks()

    .AddSqlServer(builder.Configuration.GetConnectionString("Default"));

app.MapHealthChecks("/health");
```

1 3 Background Hosted Service

```
public class EmailWorker : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken token)
    {
        while (!token.IsCancellationRequested)
        {
            Console.WriteLine("Background worker running...");
            await Task.Delay(5000);
        }
    }
}
```

Program.cs

```
builder.Services.AddHostedService<EmailWorker>();
```

1 4 Send Email (SMTP)

```
var smtp = new SmtpClient("smtp.gmail.com")
{
    Port = 587,
    Credentials = new NetworkCredential("email", "pass"),
    EnableSsl = true
};

smtp.Send("from@mail.com", "to@mail.com", "Subject", "Hello!");
```

1 5 CORS

Program.cs

```
builder.Services.AddCors(opt =>
{
    opt.AddPolicy("allowAll", p =>
        p.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader());
});

app.UseCors("allowAll");
```

 ADVANCED ASP.NET CORE (2025/2026)

1 6 Minimal APIs with Filters

Program.cs

```
app.MapGet("/hi", () => "Hello")
    .AddEndpointFilter(new LogFilter());

public class LogFilter : IEndpointFilter
{
    public async ValueTask<object> InvokeAsync(EndpointFilterInvocationContext ctx,
        EndpointFilterDelegate next)
    {
        Console.WriteLine("Request received");
        return await next(ctx);
    }
}
```

1 7 API Performance Optimization

- ✓ Use Response Caching
- ✓ Use AsNoTracking

- ✓ Use distributed caching
- ✓ Disable unnecessary Model Validation

```
builder.Services.AddResponseCaching();  
app.UseResponseCaching();
```

1 8 Distributed Caching (Redis)

Program.cs

```
builder.Services.AddStackExchangeRedisCache(opt =>  
{  
    opt.Configuration = "localhost:6379";  
});
```

Use

```
await _cache.SetStringAsync("name", "Ibrahim");  
var value = await _cache.GetStringAsync("name");
```

1 9 CQRS with MediatR Example

Command

```
public record CreateProductCommand(string Name) : IRequest<int>;
```

Handler

```
public class CreateProductHandler : IRequestHandler<CreateProductCommand, int>  
{  
    public Task<int> Handle(CreateProductCommand cmd, CancellationToken ct)  
    {  
        Console.WriteLine("Product created: " + cmd.Name);  
        return Task.FromResult(1);  
    }  
}
```

2 0 Clean Architecture Folder Structure

```
src/  
    Application/
```

Domain/

Infrastructure/

WebApi/

2 1 Onion Architecture

Layers:

- Domain
 - Application
 - Infrastructure
 - API
-

2 2 WebSockets

```
app.UseWebSockets();
```

```
app.Map("/ws", async context =>
```

```
{  
    var socket = await context.WebSockets.AcceptWebSocketAsync();  
});
```

2 3 SignalR Real-Time Dashboard

Hub

```
public class DashboardHub : Hub { }
```

Program.cs

```
builder.Services.AddSignalR();  
app.MapHub<DashboardHub>("/dashboard");
```

2 4 gRPC Service

proto file

```
service Greeter {  
    rpc SayHello (HelloRequest) returns (HelloReply);  
}
```

2 5 Multi-Tenant API

Use tenant provider:

```
public interface ITenantProvider
{
    string GetTenant();
}
```

2 6 Audit Logging with Action Filters

```
public class AuditLog : ActionFilterAttribute
{
    public override void OnActionExecuted(ActionExecutedContext ctx)
    {
        File.AppendAllText("audit.txt",
            $"{ctx.ActionDescriptor.DisplayName} executed\n");
    }
}
```

2 7 Upload Large Files (Streaming)

```
[HttpPost("big-upload")]
public async Task Upload()
{
    using var stream = new FileStream("big.dat", FileMode.Create);
    await Request.Body.CopyToAsync(stream);
}
```

2 8 API Gateway using YARP

Program.cs

```
builder.Services.AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));
app.MapReverseProxy();
```

2 9 Feature Flags

```
builder.Services.AddFeatureManagement();
```

3 0 Worker Service + Cron Jobs (.NET 8)

Using Cronos

```
var cron = CronExpression.Parse("* /5 * * * *"); // every 5 sec
```

SECTION D — LINQ & EF Core Coding

✓ 10 topics

✓ Each topic = Question + Explanation + Full Code

✓ EF Core 8 (latest patterns)

1 LINQ Queries — Filtering, Sorting, Grouping

Question

Write LINQ queries to:

1. Filter students above age 18
 2. Sort by name
 3. Group students by department
-

✓ Answer + Code

Sample Class

```
public class Student
{
    public string Name { get; set; }
    public int Age { get; set; }
    public string Department { get; set; }
}
```

Filtering

```
var adults = students.Where(s => s.Age > 18).ToList();
```

Sorting

```
var sorted = students.OrderBy(s => s.Name).ToList();
```

Grouping

```
var grouped = students
    .GroupBy(s => s.Department)
    .Select(g => new {
        Department = g.Key,
```

```
Count = g.Count(),  
Students = g.ToList()  
})  
.  
ToList();
```

 EF Core Migrations — Code Example

 Question

How do you create and apply EF Core migrations?

 Answer + Steps

1. Install EF Tools

```
dotnet tool install --global dotnet-ef
```

2. Create Migration

```
dotnet ef migrations add InitialCreate
```

3. Apply Migration

```
dotnet ef database update
```

DbContext Example

```
public class AppDbContext : DbContext  
{  
    public DbSet<Student> Students { get; set; }  
    public AppDbContext(DbContextOptions<AppDbContext> opt) : base(opt) { }  
}
```

 Query Splitting in EF Core

 Question

What is query splitting and how do you enable it?

 Answer

EF Core combines related entities into one query (which can cause Cartesian explosion).
Query splitting breaks into multiple SQL queries for performance.

Enable Query Splitting

```
var result = _ctx.Departments
    .Include(d => d.Students)
    .AsSplitQuery()
    .ToList();
```

Default mode (optional)

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.UseSqlServer(connectionString,
        o => o.UseQuerySplittingBehavior(QuerySplittingBehavior.SplitQuery));
}
```



Projection with Select



Question

Show a query selecting custom fields (DTO projection).

✓ Answer + Code

DTO

```
public class StudentDto
{
    public string Name { get; set; }
    public string Department { get; set; }
}
```

Projection

```
var list = _ctx.Students
    .Select(s => new StudentDto {
        Name = s.Name,
```

```
        Department = s.Department  
    })  
    .ToList();
```

Soft Delete Implementation

Question

Implement soft delete using IsDeleted field and global filters.

✓ Step-by-Step

1. Add Column to Entity

```
public class Student  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public bool IsDeleted { get; set; }  
}
```

2. Apply Global Query Filter

```
protected override void OnModelCreating(ModelBuilder modelBuilder)  
{  
    modelBuilder.Entity<Student>()  
        .HasQueryFilter(s => !s.IsDeleted);  
}
```

3. Replace Delete with Update

```
public async Task SoftDelete(int id)  
{  
    var s = await _ctx.Students.FindAsync(id);  
    s.IsDeleted = true;  
    await _ctx.SaveChangesAsync();  
}
```

 6 Bulk Insert using EF Core

 Question

How do you insert 10,000 records fast in EF Core?

 Answer + Code

Using EFCore.BulkExtensions

Install

dotnet add package EFCore.BulkExtensions

Bulk Insert

```
await _ctx.BulkInsertAsync(listOfStudents);
```

 7 Track vs NoTracking Example

 Question

What is AsNoTracking()? Give example.

 Answer

Tracked queries -> EF monitors changes

NoTracking -> Faster, used for read-only queries.

Tracked

```
var student = await _ctx.Students.FirstOrDefaultAsync();
```

```
student.Name = "Updated";
```

```
await _ctx.SaveChangesAsync(); // changes saved
```

No Tracking

```
var s = await _ctx.Students.AsNoTracking().FirstOrDefaultAsync();
```

```
s.Name = "Updated";
```

```
// EF will NOT save because entity is not tracked
```

 8 Custom Conventions in EF Core

Question

Create a custom EF Core convention (e.g., string max length default).

✓ Answer + Code

Convention Class

```
public class DefaultStringLengthConvention : IModelFinalizingConvention
{
    public void ProcessModelFinalizing(
        IConventionModelBuilder modelBuilder,
        IConventionContext<IConventionModelBuilder> context)
    {
        foreach (var property in modelBuilder.Metadata.GetEntityTypes()
            .SelectMany(t => t.GetProperties())
            .Where(p => p.ClrType == typeof(string)))
        {
            property.Builder.HasMaxLength(200);
        }
    }
}
```

Register Convention

```
protected override void ConfigureConventions(ModelConfigurationBuilder builder)
{
    builder.Conventions.Add(new DefaultStringLengthConvention());
}
```

EF Core Interceptors

Question

Log SQL queries using EF Core Interceptors.

✓ Answer + Code

SQL Logging Interceptor

```
public class QueryLoggingInterceptor : DbCommandInterceptor
{
    public override InterceptionResult<DbDataReader> ReaderExecuting(
        DbCommand command, CommandEventData eventData,
        InterceptionResult<DbDataReader> result)
    {
        Console.WriteLine("SQL => " + command.CommandText);
        return result;
    }
}
```

Register Interceptor

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.AddInterceptors(new QueryLoggingInterceptor());
}
```

10 Concurrency Handling (RowVersion)

? Question

Implement optimistic concurrency using RowVersion.

✓ Full Working Example

Entity with RowVersion

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }

    [Timestamp]
```

```
public byte[] RowVersion { get; set; }  
}
```

Controller Update

```
public async Task<IActionResult> Update(Product updated)  
{  
    try  
    {  
        _ctx.Products.Update(updated);  
        await _ctx.SaveChangesAsync();  
        return Ok();  
    }  
    catch (DbUpdateConcurrencyException)  
    {  
        return Conflict("Record was updated by another user.");  
    }  
}
```

SECTION E — SQL Coding (Interview + Real-World)

Each item: question → short explanation → SQL code / example.

E1. Select top N per group (e.g., top 2 salaries per dept)

```
WITH Ranked AS (  
    SELECT e.*, ROW_NUMBER() OVER (PARTITION BY Dept ORDER BY Salary DESC) rn  
    FROM Employees e  
)  
SELECT * FROM Ranked WHERE rn <= 2;
```

E2. Find duplicate rows (by columns)

```
SELECT Name, Dept, COUNT(*) cnt  
FROM Employees  
GROUP BY Name, Dept  
HAVING COUNT(*) > 1;
```

E3. Delete duplicates keeping one

```
WITH C AS (
```

C#, .NET, ASP.NET MVC, ASP.NET Core interview + non-interview coding questions

```
SELECT *, ROW_NUMBER() OVER (PARTITION BY Name, Dept ORDER BY Id) rn
FROM Employees
)
```

```
DELETE FROM C WHERE rn > 1;
```

E4. Get running totals (cumulative sum)

```
SELECT Id, OrderDate, Amount,
       SUM(Amount) OVER (ORDER BY OrderDate ROWS BETWEEN UNBOUNDED PRECEDING AND
                          CURRENT ROW) AS Cumulative
FROM Orders;
```

E5. Pagination (efficient)

```
-- SQL Server
SELECT * FROM Products
ORDER BY Id
OFFSET 50 ROWS FETCH NEXT 25 ROWS ONLY;
```

E6. Find gaps in sequence (missing invoice numbers)

```
SELECT t1.InvoiceNumber + 1 AS MissingFrom, MIN(t2.InvoiceNumber) - 1 AS MissingTo
FROM Invoices t1
LEFT JOIN Invoices t2 ON t2.InvoiceNumber > t1.InvoiceNumber
GROUP BY t1.InvoiceNumber
HAVING MIN(t2.InvoiceNumber) > t1.InvoiceNumber + 1;
```

E7. Correlated subquery vs join example

```
-- Correlated: count orders per customer
```

```
SELECT C.CustomerId,
       (SELECT COUNT(*) FROM Orders O WHERE O.CustomerId = C.CustomerId) OrderCount
FROM Customers C;
```

```
-- Join + group alternative (usually faster)
```

```
SELECT C.CustomerId, COUNT(O.Id) OrderCount
FROM Customers C
LEFT JOIN Orders O ON O.CustomerId = C.CustomerId
GROUP BY C.CustomerId;
```

E8. Pivot / Unpivot

-- Pivot example: Sales by Quarter per Region

```
SELECT Region, [Q1], [Q2], [Q3], [Q4]
FROM (
    SELECT Region, Quarter, Sales FROM SalesData
) src
PIVOT (
    SUM(Sales) FOR Quarter IN ([Q1],[Q2],[Q3],[Q4])
) p;
```

E9. Window functions — lag/lead

```
SELECT Id, Amount,
    LAG(Amount) OVER (ORDER BY OrderDate) PrevAmount,
    LEAD(Amount) OVER (ORDER BY OrderDate) NextAmount
FROM Orders;
```

E10. Efficient JOIN strategies & explain plan advice

- Use selective predicates before join.
- Ensure proper indexes on join columns.
- Use EXISTS for existence checks.
- Check execution plan; look for scans -> add indexes.

E11. Upsert (MERGE) pattern

```
MERGE INTO Products AS target
USING (VALUES (1,'Pen',10)) AS src(Id,Name,Qty)
ON target.Id = src.Id
WHEN MATCHED THEN UPDATE SET Name = src.Name, Qty = src.Qty
WHEN NOT MATCHED THEN INSERT (Id,Name,Qty) VALUES (src.Id,src.Name,src.Qty);
```

E12. Transaction isolation & deadlock handling (SQL Server pseudo)

```
BEGIN TRAN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
-- operations
COMMIT;
```

-- To handle deadlocks: capture error 1205 and retry with exponential backoff.

E13. Indexing basics (create covering index example)

```
CREATE NONCLUSTERED INDEX IX_Orders_CustomerDate ON Orders(CustomerId, OrderDate)
INCLUDE (TotalAmount);
```

E14. Normalize vs denormalize — examples & when to use

- Normalized design for OLTP; denormalize for reporting/fast reads (with ETL).
- Example: move computed totals to denormalized column maintained by triggers/ETL.

E15. Stored procedure example (insert + return id)

```
CREATE PROCEDURE InsertProduct
    @Name NVARCHAR(100), @Price DECIMAL(10,2), @NewId INT OUTPUT
AS
BEGIN
    INSERT INTO Products (Name, Price) VALUES (@Name, @Price);
    SET @NewId = SCOPE_IDENTITY();
END
```

E16. Parameter sniffing and OPTION (RECOMPILE)

- Use OPTION (RECOMPILE) or OPTIMIZE FOR UNKNOWN when parameter sniffing causes bad plans.

E17. JSON in SQL Server (read/write)

```
-- store JSON
INSERT INTO Logs(JsonData) VALUES (N'{"user":"Ibrahim","action":"login"}');

-- read
SELECT JSON_VALUE(JsonData, '$.user') AS UserName FROM Logs;
```

E18. Common table expression (CTE) recursion (hierarchy)

```
WITH OrgCTE AS (
    SELECT Id, Name, ManagerId FROM Employees WHERE ManagerId IS NULL
    UNION ALL
    SELECT e.Id, e.Name, e.ManagerId FROM Employees e
    JOIN OrgCTE o ON e.ManagerId = o.Id
)
SELECT * FROM OrgCTE;
```

E19. Performance tuning checklist

1. Identify slow queries via profiler.

2. Check indexes, missing indexes DMVs.
3. Update statistics.
4. Avoid SELECT *.
5. Optimize joins and subqueries.
6. Use proper transaction scopes.

E20. Backup & restore commands (SQL Server)

BACKUP DATABASE MyDB TO DISK = 'C:\backup\MyDB.bak';

RESTORE DATABASE MyDB FROM DISK = 'C:\backup\MyDB.bak';

SECTION F — System Design for .NET (Senior / Architect)

For each system: requirements → capacity → components → data model sketch → scaling & tradeoffs → sample endpoints & brief code snippets.

General guidelines first:

- Use Clean Architecture: UI/API → Application → Domain → Infrastructure.
 - Use containerization (Docker) + orchestration (Kubernetes).
 - Use async messaging (RabbitMQ / Kafka) for decoupling.
 - Use CDN for static assets; caching layers (Redis) for heavy read traffic.
 - Use log aggregation (ELK / Datadog) and distributed tracing (OpenTelemetry).
-

F1. Design: Billing System API

Requirements

- Create invoices, apply taxes, generate PDF, send to email, monthly reports, audit logs, multi-currency.

Capacity

- 10k invoices/day (initial), scale to 100k/day.

Components

- API (ASP.NET Core), Worker Service for PDF/email, DB (SQL), Blob storage (Azure Blob/S3), Cache (Redis), Message Broker (RabbitMQ).

Data model (key tables)

- Customers, Invoices, InvoiceItems, Payments, AuditLog.

Sample endpoints

POST /api/invoices

GET /api/invoices/{id}

POST /api/invoices/{id}/send

Important patterns

- Use idempotency keys for invoice creation.
- Use eventual consistency for background processing (PDF/email).
- Version your APIs and store calculation rules in config.

Code snippet: create invoice command (MediatR)

```
public record CreateInvoiceCmd(Guid CustomerId, List<ItemDto> Items) : IRequest<Guid>;

public class CreateInvoiceHandler : IRequestHandler<CreateInvoiceCmd, Guid> {

    public async Task<Guid> Handle(CreateInvoiceCmd req, CancellationToken ct) {

        // create invoice, save, publish event

    }

}
```

F2. Design: Inventory Management

Focus: stock reservation, high concurrency.

Key design

- Use pessimistic locking for critical stock decrements or use optimistic concurrency with retries (RowVersion).
- Use Redis to maintain hot counters for inventory read-heavy endpoints.

Endpoint

POST /api/orders -> reserve stock -> confirm or rollback.

F3. Real-Time Chat (SignalR)

Architecture

- API + SignalR Hubs, backplane (Redis) for scaling across nodes, store messages in DB or CQRS event store.

Scaling

- Use Azure SignalR Service or Redis backplane.

Hub sample

```
public class ChatHub : Hub {

    public Task Send(string user, string message) => Clients.All.SendAsync("Receive", user, message);

}
```

}

F4. Notification Service (Email + SMS)

Pattern

- REST API enqueues notification message -> Worker picks from queue -> provider-specific adapters (SMTP, Twilio).
- Implement retry/backoff (Polly).

F5. File Upload + Virus Scan

Flow

- Client uploads to pre-signed URL (S3/Azure Blob).
- After upload, Worker triggers virus scan; if clean, update DB and notify.

F6. Event-driven architecture (example: ordering)

Components

- API accepts order -> publishes OrderCreated to Kafka -> Inventory service consumes and reserves -> Payment service charges -> Order status updated.

F7. Multi-tenant SaaS

Approaches

1. Single DB, shared schema with TenantId column.
2. Single DB, separate schema per tenant.
3. Separate DB per tenant.

Tradeoffs

- Use separate DB for strong isolation; shared DB for cost efficiency.
- Use middleware to resolve tenant from host/header and set connection/tenant context.

F8. High-Availability & Disaster Recovery

- Use multiple AZs, automated backups, failover groups (SQL), replication for read replicas, health checks and circuit breakers.
-

SECTION G — Real-Time .NET Project Architecture (Practical)

This section covers the architecture and code patterns for building real-time features in .NET apps.

G1. Core building blocks

- ASP.NET Core API
- SignalR for browser real-time comms
- WebSocket fallback for low-level control
- Redis for backplane & distributed cache
- Kafka/RabbitMQ for event streaming
- Worker services for background jobs

G2. Example Project: Real-Time Stock Ticker Dashboard

Flow

- Data producer publishes quotes to Kafka.
- Backend consumer receives and pushes to connected clients via SignalR.
- Frontend subscribes to SignalR hub and renders charts.

SignalR Hub

```
public class TickerHub : Hub {  
  
    public Task Subscribe(string symbol) => Groups.AddToGroupAsync(Context.ConnectionId, symbol);  
  
    public Task Unsubscribe(string symbol) => Groups.RemoveFromGroupAsync(Context.ConnectionId, symbol);  
  
}
```

Consumer pushes to hub

```
public class QuoteConsumer {  
  
    private readonly IHubContext<TickerHub> _hub;  
  
    public async Task OnQuote(Quote q) => await _hub.Clients.Group(q.Symbol).SendAsync("Update", q);  
  
}
```

Scaling

- Use Azure SignalR or Redis backplane; deploy multiple instances behind a load balancer; use sticky sessions if not using a backplane.

G3. Connection management & resiliency

- Reconnect logic on client (exponential backoff).
- Heartbeat keepalive.

- Limit concurrent connections per user; use token-based auth for Hub connections.

G4. Data flow diagrams (text)

1. DataSource → Kafka → BackendConsumer → SignalR Hub → Browser
2. Browser → Hub (commands) → Backend (validation) → DB/Events

G5. Security

- Use JWT for Hub auth: AccessTokenProvider on client.
- Validate claims server-side in OnConnectedAsync().

SECTION H — ASP.NET Core Coding Round (MCQs + Scenarios)

I. 30 quick MCQs (with answers) — pick common interview items.

1. Q: Which middleware short-circuits the pipeline?
A: Middleware that does not call next() (e.g., authentication middleware can short-circuit).
2. Q: AsNoTracking() improves performance for read-only queries?
A: Yes.
3. Q: Default DI lifetime for AddScoped — per ?
A: HTTP request (scope).
4. Q: What is IHostedService?
A: Background service interface.
5. Q: Which package for swagger?
A: Swashbuckle.AspNetCore.
6. Q: Minimal APIs introduced in which .NET?
A: .NET 6 (refined in .NET 7/8).
7. Q: IOptions<T> is for?
A: Binding configuration to typed settings.
8. Q: Recommended way to perform HTTP retries?
A: Polly policies.
9. Q: Use DbContext in DI as?
A: Scoped.
10. Q: What's CORS short for?
A: Cross-Origin Resource Sharing.
11. Q: Task.Run inside ASP.NET Core is recommended?
A: Avoid for CPU-bound on server; use background services instead.
12. Q: How to secure minimal API endpoints?
A: Use AddAuthentication + Authorize or endpoint filters.
13. Q: What does ConfigureAwait(false) do?
A: Avoids capturing synchronization context (useful in libraries).

C#, .NET, ASP.NET MVC, ASP.NET Core interview + non-interview coding questions

14. Q: SignalR uses which transport?
A: WebSockets (primary), falls back to Server-Sent Events/Long Polling.
15. Q: Use `IAsyncEnumerable<T>` when?
A: Streaming large result sets.
16. Q: JSON serializer recommended?
A: `System.Text.Json` (default).
17. Q: For high throughput, use which DB pattern?
A: CQRS and event sourcing for high write/read separation.
18. Q: How to perform health checks?
A: `AddHealthChecks()` and `MapHealthChecks()`.
19. Q: Which scope for single instance per app?
A: Singleton.
20. Q: How to version API with header?
A: Use `ApiVersionReader` (from Microsoft API Versioning).
21. Q: Use `AsSplitQuery` when?
A: Avoid cartesian explosion from multiple collections with `Include`.
22. Q: How to prevent over-posting?
A: Use DTOs / Bind attributes.
23. Q: Is async void allowed?
A: Only for event handlers; otherwise avoid.
24. Q: How to upload streams directly to blob storage?
A: Use stream-based SDK (e.g., `BlobClient.UploadAsync(stream)`).
25. Q: Which pattern isolates domain logic?
A: Clean Architecture / Onion.
26. Q: How to set response compression?
A: `AddResponseCompression()` and `UseResponseCompression()`.
27. Q: Where to configure logging?
A: In `Program.cs` or `appsettings.json` (providers).
28. Q: How to do integration tests for APIs?
A: `WebApplicationFactory<TEntryPoint>` from `Microsoft.AspNetCore.Mvc.Testing`.
29. Q: Rate limiting in .NET 8 built-in?
A: Yes, `AddRateLimiter()`.
30. Q: How to implement feature flags?
A: `Microsoft.FeatureManagement` or external like `LaunchDarkly`.

II. Practical coding scenarios (with solutions) — 6 common rounds.

Scenario H1 — Implement paged, searchable product list API (with caching)

Endpoint: GET /api/products?page=1&pageSize=20&q=pen

Solution highlights

- Accept page, pageSize, q DTO.
- Use AsNoTracking(), Where(...), .Skip().Take(), Select() DTO projection.
- Add response caching / Redis cache key products:q:page:pageSize.

Code (controller method)

[HttpGet]

```
public async Task<ActionResult> List([FromQuery] string q, int page=1, int pageSize=20){
    string cacheKey = $"products:{q}:{page}:{pageSize}";
    var cached = await _cache.GetStringAsync(cacheKey);
    if(cached != null) return Content(cached, "application/json");

    var query = _ctx.Products.AsNoTracking()
        .Where(p => string.IsNullOrEmpty(q) || p.Name.Contains(q));
    var total = await query.CountAsync();
    var items = await query.OrderBy(p => p.Id)
        .Skip((page-1)*pageSize)
        .Take(pageSize)
        .Select(p => new { p.Id, p.Name, p.Price })
        .ToListAsync();

    var result = new { total, items };
    await _cache.SetStringAsync(cacheKey, JsonSerializer.Serialize(result), new
    DistributedCacheEntryOptions { AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(2) });
    return Ok(result);
}
```

Scenario H2 — Implement JWT login + refresh tokens (concise)

- On login: validate credentials -> issue JWT + persist refresh token with expiry.
- Refresh endpoint: validate refresh token exists, not expired, issue new JWT & rotate refresh token.

Refresh token generation

```
var refresh = Convert.ToBase64String(RandomNumberGenerator.GetBytes(64));
```

Scenario H3 — Implement background file processor using Worker Service

- API uploads file to blob, enqueues message (RabbitMQ), Worker consumes, processes and updates DB.

Worker skeleton

```
public class FileProcessor : BackgroundService {  
  
    protected override async Task ExecuteAsync(CancellationToken ct) {  
  
        while(!ct.IsCancellationRequested){  
  
            var msg = await _queue.DequeueAsync(ct);  
  
            // process file  
  
        }  
  
    }  
  
}
```

Scenario H4 — Implement optimistic concurrency (RowVersion)

- Add [Timestamp] byte[] RowVersion property.
- Catch DbUpdateConcurrencyException and return 409.

Scenario H5 — Write an endpoint that streams large CSV result (IEnumerable)

```
[HttpGet("export")]  
  
public async IEnumerable<string> Export(){  
  
    await foreach(var row in _ctx.Products.AsNoTracking().AsAsyncEnumerable()){  
  
        yield return $"{row.Id},{row.Name},{row.Price}\n";  
  
    }  
  
}
```

Scenario H6 — Build SignalR notification endpoint + hub

- Controller triggers IHubContext<NotifHub>.Clients.User(userId).SendAsync("Notify", payload);

100 .NET Interview Questions (2025–2026) — Q & concise A

I grouped by topic. Short answers focused on interviews.

Core C# (1–20)

1. Q: Difference between ref and out? — A: ref requires initialized variable; out doesn't; both pass by reference.
2. Q: async/await — A: async marks method; await yields control until Task completes.

C#, .NET, ASP.NET MVC, ASP.NET Core interview + non-interview coding questions

3. Q: Task vs Thread? — A: Task is abstraction for asynchronous work (schedulers), Thread is OS thread.
4. Q: IEnumerable vs IQueryable? — A: IEnumerable executes in memory; IQueryable builds expression tree for provider (deferred).
5. Q: Boxing/unboxing? — A: Converting value type to object (boxing) and back (unboxing) — performance cost.
6. Q: volatile keyword? — A: Prevents compiler optimization re ordering; ensures reads/writes go to memory.
7. Q: lock vs Monitor? — A: lock is syntactic sugar around Monitor.Enter/Exit.
8. Q: readonly vs const? — A: const compile-time constant; readonly runtime constant set in ctor.
9. Q: IDisposable pattern? — A: Implement Dispose, optional finalizer, GC.SuppressFinalize(this).
10. Q: sealed class? — A: Prevents inheritance.
11. Q: Covariance/Contravariance? — A: out enables covariance; in contravariance in generics.
12. Q: Span<T>? — A: Stack-allocated or memory-slice type to avoid allocations.
13. Q: record vs class? — A: record provides value-based equality and immutability helpers.
14. Q: struct best practices? — A: Small, immutable, avoid boxing.
15. Q: Expression<Func<T,bool>> vs Func<T,bool>? — A: Expression build tree (used by EF); Func is compiled delegate.
16. Q: GC.Collect() — A: Avoid calling manually; let runtime handle GC.
17. Q: Nullable<T> usage? — A: T? for value types to allow null.
18. Q: Mutex vs Semaphore? — A: Mutex across processes; Semaphore limits concurrent threads.
19. Q: Interlocked operations? — A: Atomic operations for simple concurrency-safe updates.
20. Q: Differences between StringBuilder and string? — A: String immutable; StringBuilder for many modifications.

.NET Core / ASP.NET Core (21–50)

21. Q: How is Middleware ordered? — A: Order added in Program.cs is execution order.
22. Q: What is Kestrel? — A: Cross-platform web server for ASP.NET Core.
23. Q: IServiceCollection lifetimes? — A: Singleton, Scoped, Transient.
24. Q: How to configure Logger? — A: Use ILoggingBuilder in Program or appsettings.
25. Q: What is HealthChecks? — A: Built-in health check framework.
26. Q: Why use IOptions<T>? — A: Strongly-typed configuration binding.

C#, .NET, ASP.NET MVC, ASP.NET Core interview + non-interview coding questions

- 27. Q: How to do dependency injection for EF? — A: `AddDbContext<T>()`.
- 28. Q: How to do integration testing? — A: `WebApplicationFactory<TEntryPoint>`.
- 29. Q: How to implement authentication? — A: JWT, cookies, OAuth, IdentityServer.
- 30. Q: Describe `UseRouting` vs `UseEndpoints`? — A: `UseRouting` builds route data; `UseEndpoints` executes endpoint.
- 31. Q: What is endpoint routing? — A: Unified routing system from ASP.NET Core 3.0.
- 32. Q: How to log exceptions globally? — A: Exception middleware or `UseExceptionHandler`.
- 33. Q: What is gRPC? — A: High-performance RPC framework using Protobuf.
- 34. Q: When use Kestrel behind reverse proxy? — A: Production use with Nginx/IIS for TLS, routing.
- 35. Q: What is minimal API? — A: Lightweight low-boilerplate API in `Program.cs`.
- 36. Q: How to implement CORS? — A: `AddCors().AddPolicy()` and `UseCors()`.
- 37. Q: How to support SignalR? — A: `AddSignalR()` and `MapHub<>()`.
- 38. Q: How to use configuration secrets in production? — A: Azure KeyVault or environment variables.
- 39. Q: How to handle file uploads? — A: `IFormFile`, stream to blob to avoid buffering.
- 40. Q: What is Data Protection in ASP.NET? — A: Cryptographic APIs for secure storage of keys and tokens.
- 41. Q: How to do request validation? — A: Model binding + `DataAnnotations` or `FluentValidation`.
- 42. Q: What is response caching? — A: Cache responses via middleware or headers.
- 43. Q: How to implement rate limiting? — A: .NET 8 `AddRateLimiter()` or API gateway.
- 44. Q: How to do graceful shutdown? — A: `IHostApplicationLifetime` and cancellation tokens.
- 45. Q: How to configure Kestrel limits? — A: `KestrelServerOptions` (max request body size, timeouts).
- 46. Q: What is HealthChecks UI? — A: Dashboard for health endpoints (third-party).
- 47. Q: How to use distributed cache? — A: `AddStackExchangeRedisCache`.
- 48. Q: What is `BackgroundService`? — A: Base class for hosted background workers.
- 49. Q: How to manage secrets locally? — A: `dotnet user-secrets`.
- 50. Q: How to enable HTTPS redirection? — A: `app.UseHttpsRedirection()`.

Entity Framework Core (51–70)

- 51. Q: `AsNoTracking()` purpose — A: Read-only performance improvement.
- 52. Q: `Migrations` vs `EnsureCreated` — A: `Migrations` for schema evolution, `EnsureCreated` for dev/testing.

- 53. Q: Concurrency tokens (RowVersion) — A: For optimistic concurrency.
- 54. Q: Bulk operations — A: Use third-party libs (EFCore.BulkExtensions).
- 55. Q: Query splitting — A: AsSplitQuery() to avoid cartesian explosion.
- 56. Q: ChangeTracker behavior — A: Tracks states of entities for SaveChanges.
- 57. Q: Lazy loading in EF Core? — A: Use proxies or explicit loading, enabled via packages.
- 58. Q: Interceptors — A: IDbCommandInterceptor, ISaveChangesInterceptor for cross-cutting concerns.
- 59. Q: Projection with Select — A: Avoid full entity materialization for perf.
- 60. Q: NoSQL with EF Core? — A: EF Core providers exist for some NoSQL but limited.
- 61. Q: How to seed data? — A: modelBuilder.Entity<T>().HasData(...) in OnModelCreating.
- 62. Q: Migrations in CI/CD? — A: Apply migrations at deploy time or use separate migration step.
- 63. Q: Eager vs Lazy vs Explicit loading — A: Eager (Include), Lazy (proxies), Explicit (LoadAsync).
- 64. Q: Tracking query memory leak cause — A: Long-lived DbContext + tracked entities growth. Use scoped DbContext.
- 65. Q: How to handle N+1 problem? — A: Use Include or projection; check generated SQL.
- 66. Q: Soft delete pattern — A: Query filters with IsDeleted.
- 67. Q: Use stored procedures with EF Core? — A: Use FromSqlRaw and ExecuteSqlRaw.
- 68. Q: When to use raw SQL? — A: Complex queries or performance-critical cases.
- 69. Q: How to map value objects? — A: Owned entity types.
- 70. Q: How to use transactions? — A: IDbContextTransaction or TransactionScope.

Architecture & Design (71–90)

- 71. Q: What is CQRS? — A: Command Query Responsibility Segregation.
- 72. Q: Event Sourcing? — A: Persist events as source of truth, rebuild state by replay.
- 73. Q: Circuit Breaker pattern? — A: Prevent cascading failures with retry/backoff (Polly).
- 74. Q: Saga pattern? — A: Manage long-running distributed transactions through compensating actions.
- 75. Q: API Gateway role? — A: Single endpoint, routing, auth, rate limiting.
- 76. Q: When to use message broker? — A: For async decoupling, scaling, resilience.
- 77. Q: How to design for scale? — A: Stateless services, scale out, caching, partitioning.
- 78. Q: Idempotency? — A: Designing endpoints safe for retries, use idempotency keys.
- 79. Q: Bulkhead isolation? — A: Isolate failures by limiting resource groups.
- 80. Q: Observability stack? — A: Logs, metrics, traces (ELK/Prometheus/OpenTelemetry).

C#, .NET, ASP.NET MVC, ASP.NET Core interview + non-interview coding questions

- 81. Q: Multi-tenant concerns? — A: Tenant resolution, isolation, per-tenant config.
- 82. Q: Data partitioning strategies? — A: Range, hash, list partitioning, sharding.
- 83. Q: Scaling DB read/write? — A: Read replicas, CQRS, partitioning.
- 84. Q: Blue/Green deployments? — A: Zero-downtime deployments via two environments.
- 85. Q: Canary releases? — A: Gradual rollout to subset of users.
- 86. Q: Secrets management? — A: Vaults/KeyVault and least-privilege.
- 87. Q: SSO vs OAuth? — A: SSO is single sign-on UX; OAuth is authorization protocol.
- 88. Q: Microservices tradeoffs? — A: Independent deploy vs complexity/operational overhead.
- 89. Q: API throttling design? — A: Token buckets, fixed windows; implement at gateway.
- 90. Q: Data migration strategies? — A: Backfill, rolling migration, dual-write, feature flags.

Testing, DevOps & Misc (91–100)

- 91. Q: Unit vs Integration tests? — A: Unit tests small isolated; integration tests components together.
- 92. Q: Use Moq or NSubstitute? — A: Both are mocking libs; preference-based.
- 93. Q: CI/CD tools? — A: GitHub Actions, Azure Pipelines, GitLab CI.
- 94. Q: Containerize app? — A: Create Dockerfile, multi-stage builds, push to registry.
- 95. Q: Health checks in Kubernetes? — A: Liveness and readiness probes.
- 96. Q: Use feature toggles? — A: For safe releases; store flags centrally.
- 97. Q: How to secure microservices? — A: mTLS, OAuth2, mutual auth, least privilege.
- 98. Q: What is OpenAPI? — A: Specification for describing RESTful APIs (Swagger).
- 99. Q: How to profile performance? — A: Use dotnet-trace, PerfView, Application Insights.
- 100. Q: How to migrate .NET Framework to .NET Core? — A: Port libs, update NuGet, test, move to csproj SDK style, handle Windows-only APIs.

SYSTEM DESIGN — FULL PACK (2025/2026 Updated)

Contains **10 complete system design answers**.

Design a Billing System API

(Invoice creation, taxes, PDF, email, payments)

Requirements

- Create and manage invoices
 - Tax calculation
 - Save PDF
 - Email invoice automatically
 - Track invoice status (Draft → Sent → Paid)
 - Payment integration
-

High Level Architecture

Client → API Gateway → Billing API → DB

| → PDF Service

| → Email Service

| → Payment Service

Database Design

Invoice

Column	Type
Id	int
InvoiceNo	nvarchar
CustomerId	int
TotalAmount	decimal
TaxAmount	decimal
Status	tinyint

Column	Type
CreatedDate	datetime

InvoiceItems

Column	Type
Id	int
InvoiceId	int
Description	nvarchar
Quantity	int
Price	decimal

API Endpoints

POST /invoice/create

GET /invoice/{id}

PUT /invoice/pay/{id}

GET /invoice/download/{id}

Core Logic — Tax + Total Calculation

```
public Invoice CreateInvoice(Invoice invoice)
{
    invoice.TaxAmount = invoice.Items.Sum(i => i.Price * i.Quantity) * 0.18m;
    invoice.TotalAmount = invoice.Items.Sum(i => i.Price * i.Quantity) + invoice.TaxAmount;
    invoice.Status = InvoiceStatus.Draft;

    _context.Invoices.Add(invoice);
    _context.SaveChanges();

    return invoice;
}
```

PDF Generation (OpenXML or iText)

```
public byte[] GeneratePdf(Invoice invoice)
{
```

```
using (MemoryStream ms = new MemoryStream())
{
    var writer = new PdfWriter(ms);
    var pdf = new PdfDocument(writer);
    var doc = new Document(pdf);

    doc.Add(new Paragraph($"Invoice No: {invoice.InvoiceNo}"));
    doc.Add(new Paragraph($"Total: {invoice.TotalAmount}"));

    doc.Close();
    return ms.ToArray();
}
```

Email Service (SMTP)

```
public async Task SendInvoiceEmail(string email, byte[] pdf)
{
    MailMessage msg = new();
    msg.To.Add(email);
    msg.Subject = "Your Invoice";
    msg.Attachments.Add(new Attachment(new MemoryStream(pdf), "Invoice.pdf"));

    using SmtpClient smtp = new("smtp.gmail.com");
    smtp.Port = 587;
    smtp.Credentials = new NetworkCredential("test@gmail.com", "app-password");
    smtp.EnableSsl = true;

    await smtp.SendMailAsync(msg);
}
```

✓ 2 Design an Inventory Management System

📌 Requirements

- Maintain stock
 - Purchase / Sale updates
 - Low stock alerts
 - Stock aging report
 - Multi-warehouse
-

📌 Architecture

API → Inventory Service → DB

| → Notification Service

| → Report Service

📌 Database Tables

Products

(Id, Name, Category, SKU)

Stock

(Id, ProductId, Qty, WarehouseId)

Transactions

(Id, ProductId, QtyChange, Type: Purchase/Sale)

📌 Real Stock Update Logic

```
public async Task UpdateStock(int productId, int qtyChange)
{
    var stock = await _ctx.Stock.FirstAsync(x => x.ProductId == productId);

    stock.Qty += qtyChange; // + for purchase, - for sale
    await _ctx.SaveChangesAsync();

    if (stock.Qty < 10)
```

```
await _notify.LowStockAlert(productId, stock.Qty);  
}
```

 3 Design Real-Time Chat using SignalR

 Architecture

Client → SignalR Hub → Redis Backplane → Other Nodes

 Hub Code

```
public class ChatHub : Hub  
{  
    public async Task SendMessage(string user, string message)  
    {  
        await Clients.All.SendAsync("ReceiveMessage", user, message);  
    }  
}
```

 Program.cs

```
builder.Services.AddSignalR()  
    .AddStackExchangeRedis("localhost:6379");
```

 4 Design Notification Service (Email + SMS)

 Architecture

Client → Notification API → Queue (RabbitMQ) → Workers → Email/SMS Providers

 Publish Message to Queue

```
public void Publish(NotificationMessage msg)
```



```
{  
    var json = JsonSerializer.Serialize(msg);  
    var body = Encoding.UTF8.GetBytes(json);  
  
    _channel.BasicPublish("", "notification_queue", null, body);  
}
```

Worker Consuming Queue

```
var consumer = new EventingBasicConsumer(channel);  
consumer.Received += async (model, ea) =>  
{  
    var json = Encoding.UTF8.GetString(ea.Body.ToArray());  
    var msg = JsonSerializer.Deserialize<NotificationMessage>(json);  
  
    if (msg.Type == "Email")  
        await _email.Send(msg);  
    else  
        await _sms.Send(msg);  
};
```

5 Design High Performance Report API

Requirements

- Heavy SQL queries
 - Millions of rows
 - Paginated API
 - Caching
 - Background report generation
-

Architecture

Client → Report API → Cache → DB

↳ Background Job → Excel/PDF

Async Report Generation (Hangfire)

```
BackgroundJob.Enqueue(() => _report.Create(reportRequest));
```

Cached Query

```
var data = await _cache.GetOrCreateAsync("sales-report", entry =>
{
    entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(10);
    return _ctx.Sales
        .Where(x => x.Date >= start && x.Date <= end)
        .ToListAsync();
});
```

6 Design File Upload + Virus Scan Service

Architecture

Client → API → Storage → Virus Scan Worker → Mark Safe/Unsafe

Save File

```
var filePath = Path.Combine("uploads", file.FileName);
using var stream = new FileStream(filePath, FileMode.Create);
await file.CopyToAsync(stream);
```

Enqueue file for virus scan

```
PublishToQueue(new ScanRequest { FilePath = filePath });
```

✔ 7 Design Token Authentication Service

📌 Architecture

Client → Auth API → JWT Service → DB

📌 Generate JWT

```
public string GenerateToken(string userId)
{
    var claims = new[]
    {
        new Claim(ClaimTypes.NameIdentifier, userId)
    };

    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: "myapp",
        audience: "myapp",
        claims: claims,
        expires: DateTime.Now.AddHours(2),
        signingCredentials: creds);

    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

✔ 8 Design Multi-Tenant SaaS Application

📌 Tenant Models

- **Single Database, Shared Schema** (TenantId column)

- Single Database per Tenant
- Isolated DB + Isolated Services (Enterprise SaaS)

 Best Approach (2025)

→ **Shared Database + Row Level Filtering**

 Tenant Filter

protected override void OnModelCreating(ModelBuilder modelBuilder)

```
{  
    modelBuilder.Entity<Invoice>()  
        .HasQueryFilter(i => i.TenantId == _tenantProvider.TenantId);  
}
```

 9 Event-Driven Architecture using RabbitMQ

 Architecture

Order API → Publish EVENT → RabbitMQ → Inventory Worker → Email Worker → Billing Worker

 Publish Event

```
channel.BasicPublish(  
    exchange: "order.exchange",  
    routingKey: "order.created",  
    body: Encoding.UTF8.GetBytes(json));
```

 Consume Event

```
consumer.Received += (sender, ea) =>  
{  
    var msg = Encoding.UTF8.GetString(ea.Body.ToArray());  
    var order = JsonSerializer.Deserialize<OrderEvent>(msg);
```

```
_inventory.Update(order);  
};
```

10 Distributed Caching Design

Architecture

API → Memory Cache → Redis Cache → DB

Redis Cache Example

```
public async Task<Product> GetProduct(int id)  
{  
    var cacheKey = $"product:{id}";  
  
    var cached = await _redis.GetStringAsync(cacheKey);  
    if (cached != null)  
        return JsonSerializer.Deserialize<Product>(cached);  
  
    var product = await _ctx.Products.FindAsync(id);  
  
    if (product != null)  
        await _redis.SetStringAsync(cacheKey, JsonSerializer.Serialize(product));  
  
    return product;  
}
```